



ECR Integration with Kubernetes

Understanding Kubernetes for Orchestration

Introduction

Amazon Elastic Container Registry (ECR) is a fully managed container image registry provided by AWS. It allows developers to **store, manage, and deploy Docker container images securely**.

Integrating **ECR with Kubernetes** enables Kubernetes Pods to **pull images from a private ECR repository** securely.

This guide will cover:

- **What is ECR and why use it?**
 - **How to push and pull images from ECR**
 - **Integrating ECR with Kubernetes Deployments**
 - **Common issues and FAQs**
-

Why Use Amazon ECR?

- **Fully Managed** – No need to set up and maintain container registries.
 - **Secure** – Uses **AWS IAM roles** for access control.
 - **Faster Deployments** – Optimized for AWS-based Kubernetes clusters.
 - **Automated Image Scanning** – Detects vulnerabilities in container images.
-

Step 1: Create an ECR Repository

To create a new **private ECR repository**, run:

```
aws ecr create-repository --repository-name my-app
```



Retrieve the repository URI:

```
aws ecr describe-repositories --query "repositories[].repositoryUri"
```

Step 2: Authenticate Docker with ECR

Get the ECR login token and authenticate Docker:

```
aws ecr get-login-password | docker login --username AWS --password-stdin  
<aws_account_id>.dkr.ecr.<region>.amazonaws.com
```

Step 3: Push a Docker Image to ECR

1. **Tag the image with the ECR repository URI:**

```
docker tag my-app:latest  
<aws_account_id>.dkr.ecr.<region>.amazonaws.com/my-app:latest
```

2. **Push the image to ECR:**

```
docker push  
<aws_account_id>.dkr.ecr.<region>.amazonaws.com/my-app:latest
```

Step 4: Deploy Kubernetes Workloads Using ECR Images

Create a Kubernetes Deployment using an image from ECR:

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: my-app  
spec:
```



```
replicas: 2
selector:
  matchLabels:
    app: my-app
template:
  metadata:
    labels:
      app: my-app
spec:
  containers:
    - name: my-app-container
      image: <aws_account_id>.dkr.ecr.<region>.amazonaws.com/my-app:latest
      ports:
        - containerPort: 80
```

Apply the deployment:

```
kubectl apply -f deployment.yaml
```

Common Issues and Troubleshooting

Problem	Solution
AccessDeniedException	Ensure IAM role has permissions to access ECR
Pod cannot pull image	Verify imagePullSecrets in Deployment



Problem	Solution
Authentication failure	Re-run <code>aws ecr get-login-password</code>

Conclusion

Integrating **ECR with Kubernetes** provides a **secure and efficient** way to manage container images in AWS.

👉 Try **pushing an image to ECR** and **deploying it in your EKS cluster**!